Lesson 15 — Actuator Limits, Torque Saturation and Anti-Windup

Module 3 · Robot Dynamics & Control · Completed 21 October 2026

Learning outcomes

• Explain actuator torque saturation and why requested motor torque may differ from actual torque.

• Simulate a gravity-loaded joint with PD control and a maximum applied torque.

• Recognize integral windup, its causes, and basic prevention strategies.

1. Torque saturation

Real actuators cannot apply arbitrary torque. For symmetric limits:

```
-tau_max <= tau_actual <= tau_max
tau_actual = clip(tau_requested, -tau_max, +tau_max)
```

Example: requested +12 N·m, torque limit ±5 N·m → applied +5 N·m.

2. Dynamics and controller

Single-joint model (angle measured counterclockwise from horizontal):

```
I * theta_ddot + b * theta_dot + m*g*L*cos(theta) = tau_actual
tau_requested = Kp*(theta_target-theta) - Kd*theta_dot + m*g*L*cos(theta)
```

The controller requests gravity compensation, but the actual motor output is clipped. Hence compensation may be insufficient when the torque limit is low.

3. Integral windup

For PID control, the integral state accumulates position error:

```
error_integral(t) = integral(error(s) ds, s=0..t)
tau_PID = Kp*error + Ki*error_integral + Kd*error_dot
```

When applied torque saturates, the integral may continue to accumulate and cause excessive overshoot later. A simple introductory anti-windup method is integral clamping:

```
error_integral = clip(error_integral, -limit, +limit)
```

More principled methods include conditional integration and back-calculation. Note: the Lesson 15 Python experiment uses PD, not an integral controller, so it demonstrates saturation rather than windup experimentally.

4. Reproducible Python experiment

Save as lesson_15_torque_saturation.py and run using python lesson_15_torque_saturation.py. Requires NumPy and Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

I, b = 0.5, 0.2
m, L, g = 1.0, 0.5, 9.81
Kp, Kd = 12.0, 3.0
target = np.deg2rad(90.0)
dt, duration = 0.005, 4.0
steps = int(duration / dt)

def simulate(torque_limit):
    theta, omega = 0.0, 0.0
```

```
    angles = [0.0]
    saturated_steps = 0
    for _ in range(steps):
        error = target - theta
        requested = Kp*error - Kd*omega + m*g*L*np.cos(theta)
        applied = np.clip(requested, -torque_limit, torque_limit)
        if abs(requested) > torque_limit:
            saturated_steps += 1
        alpha = (applied - b*omega - m*g*L*np.cos(theta)) / I
        omega += alpha*dt
        theta += omega*dt
        angles.append(np.rad2deg(theta))
    return np.array(angles), saturated_steps

unlimited, _ = simulate(float('inf'))
limited, saturated = simulate(5.0)
time = np.linspace(0, duration, steps + 1)
print(f'Unlimited final angle: {unlimited[-1]:.3f} deg')
print(f'Limited final angle:   {limited[-1]:.3f} deg')
print(f'Saturated time steps: {saturated}')
assert saturated > 0
assert np.all(np.isfinite(limited))
print('Verification: PASSED')
plt.plot(time, unlimited, label='Unlimited motor')
plt.plot(time, limited, label='Torque limit: 5 N·m')
plt.axhline(90, linestyle='--', label='Target')
plt.xlabel('Time (s)'); plt.ylabel('Joint angle (deg)')
plt.title('Lesson 15: Torque Saturation')
plt.grid(True); plt.legend(); plt.tight_layout()
plt.savefig('lesson_15_torque_saturation.png', dpi=160)
plt.show()
```

5. Observations and verification

User reported that the Python experiment PASSED, the two responses differed, and saturation occurred. No exact numerical trajectory or screenshot was provided; none is claimed here. The assertion verifies saturated steps and finite values, not guaranteed convergence or stability across all settings.

6. Assessment record

• Python: PASSED (reported).

• Torque limit question: B, +5 N·m — correct.

• Windup question: B, excessive integral accumulation — correct.

• Result: 3/3; Lesson 15 completed.

7. Practical limitations

This is a simple rigid single-joint model with semi-implicit Euler integration, ideal state measurements, viscous damping, no actuator delay, and no thermal or power model. Robust physical implementation needs torque/speed limits, controller anti-windup, sensor feedback, and safety constraints.